Tecnológico de Monterrey

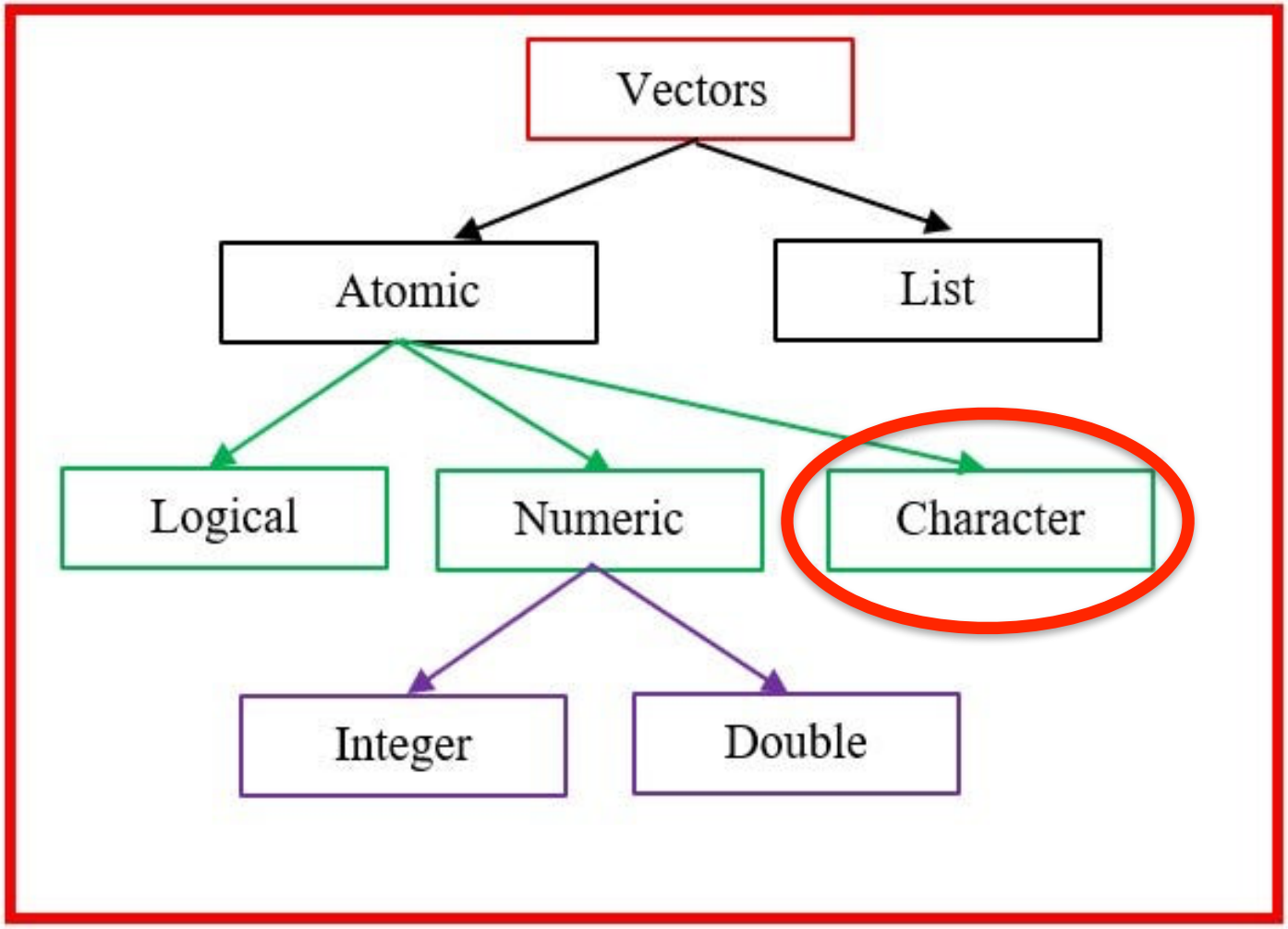
11. Texto con *stringr*



Ciencia de datos II

Jorge Juvenal Campos Ferreira

juvenal.campos@tec.mx



NULL

https://medium.com/analytics-vidhya/the-ultimate-beginners-guide-to-data-types-in-r-ac4358985150

Definiciones básicas



⭐Texto

Un texto (o cadena, *string*) es una **secuencia finita de símbolos tomados de un alfabeto.**

Dado un alfabeto Σ (un conjunto finito y no vacío de símbolos), **una cadena** es cualquier secuencia finita $s = s_1, s_2, \ldots s_n$ donde cada $s_i \in \Sigma$.

La longitud de dicha cadena es n, y el caso n = 0 es la cadena vacía (ε, que en R es ""). El conjunto de todas las cadenas posibles se escribe Σ*.

Cuenta con las siguientes propiedades:

- **El orden importa:** "roma" y "amor" tienen los mismos símbolos, pero son cadenas distintas.
- **La longitud es variable:** un tipo de dato double siempre ocupa 8 bytes, mientras que una cadena puede tener cualquier longitud.
- **No tiene magnitud:** su operación natural es la concatenación, no la aritmética. Por eso "1" + "2" da error en R, aunque los símbolos sean dígitos.







⭐vector tipo character

Secuencia ordenada y homogénea en la que cada elemento es, o una cadena de texto de longitud arbitraría, o una cadena vacía (“”), o el valor faltante de tipo caracter (*NA_character*).

⭐Carácter

Es la unidad mínima de texto que un sistema puede almacenar y representar. A cada carácter le corresponde un código numérico, por ejemplo, la “ñ” e U+00F1.

⭐Unicode

Estándar internacional que asigna a cada carácter de **TODOS** los sistemas de escritura un número entero único, llamado punto de código.

Formalmente es un conjunto de caracteres codificado: una función que mapea caracteres abstractos a enteros en el rango de U+0000 a U+10FFFF (1,114,112 posiciones posibles, de las cuales están asignadas unas 150 a 160 mil, cifra que crece con cada versión). Lo mantiene el Consorcio Unicode desde 1991.

Más conceptos

⭐Encoding (codificación)

Es la regla que convierte puntos de código en bytes y viceversa.

Ejemplo: Unicode dice que "ñ" es el número 241; la codificación decide como se escribe ese 241 en la memoria.

Antes (*ahora mucho menos)* cada computadora tenía que configurar el encoding para saber con qué conjunto de caracteres tenía que escribir. Ahora se da menos. (

⭐UTF-8 (*Unicode Transformation Format*)

Es la codificación de Unicode más utilizada (casi toda la web la utiliza). Es de longitud variable, compatible con ASCII (los mismos valores en ambos encodings para las letras que se usan en el inglés), cubre todo el catálogo Unicode (incluyendo kanjis y emojis) y es el estándar de R desde la versión 4.2.

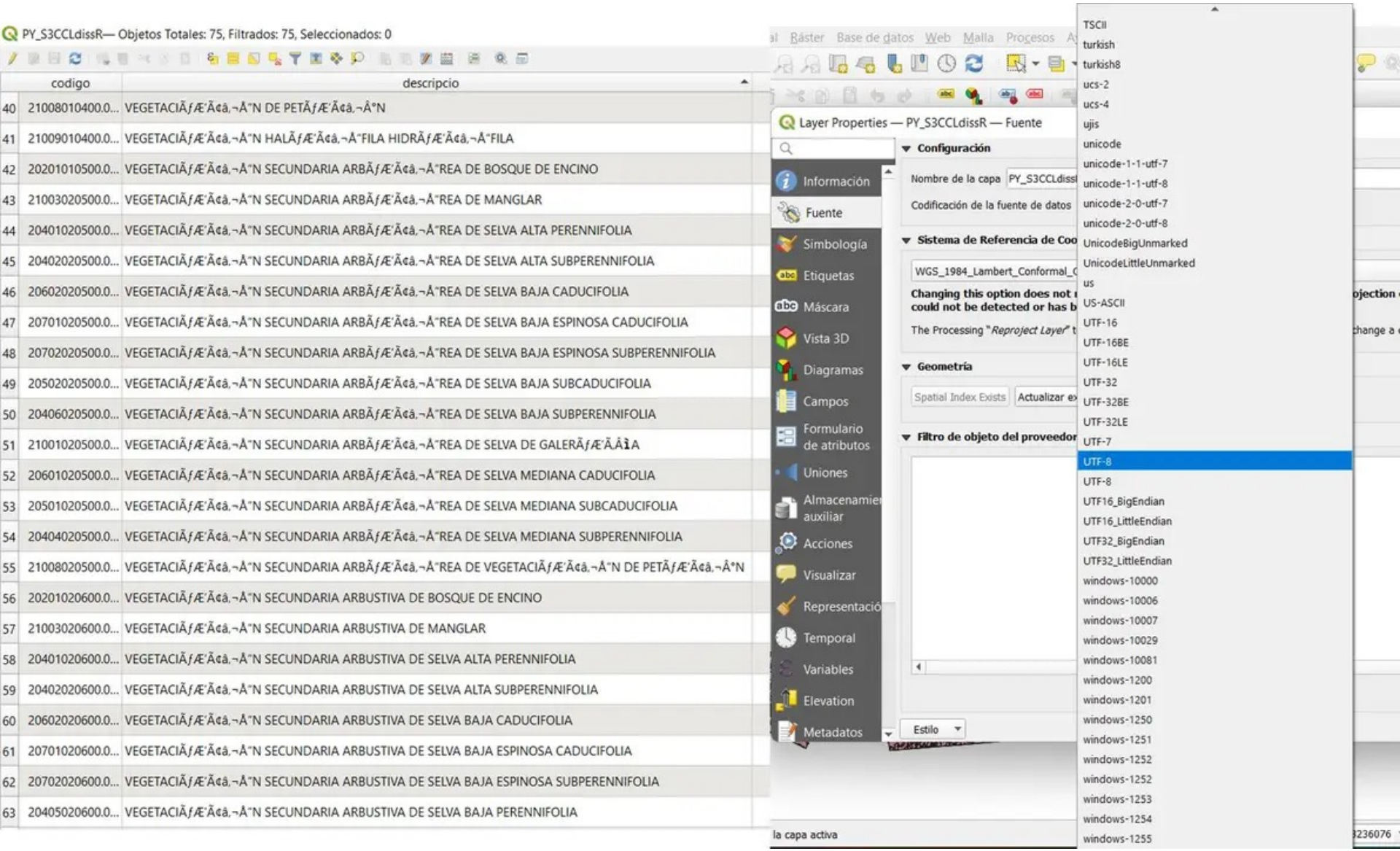
Rango	Bytes	Ejemplos
U+0000 a U+007F	1	a, Z, 7, ?
U+0080 a U+07FF	2	ñ, é, ü
U+0800 a U+FFFF	3	日, €
U+10000 en adelante	4	emojis



Es importante saber que no todos los textos ni todos los programas que usamos utilizan el mismo encoding.

Por **ejemplo**, la letra "**ñ**" en UTF-8, que es una letra representada por la suma de dos bytes (n y ~), en un encoding distinto (latin-1) se vería **Ã±**.

(Esto es importante tomar en cuenta al abrir archivos de gobierno, por ejemplo, que luego suelen usar otros encodings).


PY_S3CCLdissR— Objetos Totales: 75, Filtrados: 75, Seleccionados: 0
codigo
descripcio
40 21008010400.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N DE PETÃƒÆ'Ã¢â‚¬Â"N
41 21009010400.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N HALÃƒÆ'Ã¢â‚¬Â"FILA HIDRÃƒÆ'Ã¢â‚¬Â"FILA
42 20201010500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE BOSQUE DE ENCINO
43 21003020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE MANGLAR
44 20401020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA ALTA PERENNIFOLIA
45 20402020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA ALTA SUBPERENNIFOLIA
46 20602020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA BAJA CADUCIFOLIA
47 20701020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA BAJA ESPINOSA CADUCIFOLIA
48 20702020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA BAJA ESPINOSA SUBPERENNIFOLIA
49 20502020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA BAJA SUBCADUCIFOLIA
50 20406020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA BAJA SUBPERENNIFOLIA
51 21001020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA DE GALERÃƒÆ'Ã‚Â¡A
52 20601020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA MEDIANA CADUCIFOLIA
53 20501020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA MEDIANA SUBCADUCIFOLIA
54 20404020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE SELVA MEDIANA SUBPERENNIFOLIA
55 21008020500.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBÃƒÆ'Ã¢â‚¬Â"REA DE VEGETACIÃƒÆ'Ã¢â‚¬Â"N DE PETÃƒÆ'Ã¢â‚¬Â"N
56 20201020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE BOSQUE DE ENCINO
57 21003020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE MANGLAR
58 20401020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE SELVA ALTA PERENNIFOLIA
59 20402020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE SELVA ALTA SUBPERENNIFOLIA
60 20602020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE SELVA BAJA CADUCIFOLIA
61 20701020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE SELVA BAJA ESPINOSA CADUCIFOLIA
62 20702020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE SELVA BAJA ESPINOSA SUBPERENNIFOLIA
63 20405020600.0... VEGETACIÃƒÆ'Ã¢â‚¬Â"N SECUNDARIA ARBUSTIVA DE SELVA BAJA PERENNIFOLIA
Ráster
Base de datos
Web
Malla
Procesos
Layer Properties — PY_S3CCLdissR — Fuente
Información
Fuente
Simbología
Etiquetas
Máscara
Vista 3D
Diagramas
Campos
Formulario de atributos
Uniones
Acciones
Visualizar
Temporal
Variables
Elevation
Metadatos
Configuración
Nombre de la capa
Codificación de la fuente de datos
Sistema de Referencia de Coo
WGS_1984_Lambert_Conformal_
Changing this option does not
could not be detected or has b
The Processing "Reproject Layer"
Geometría
Spatial Index Exists
Actualizar ex
Filtro de objeto del proveedor
Estilo
la capa activa
TSCII
turkish
turkish8
ucs-2
ucs-4
ujis
unicode
unicode-1-1-utf-7
unicode-1-1-utf-8
unicode-2-0-utf-7
unicode-2-0-utf-8
UnicodeBigUnmarked
UnicodeLittleUnmarked
us
US-ASCII
UTF-16
UTF-16BE
UTF-16LE
UTF-32
UTF-32BE
UTF-32LE
UTF-7
UTF-8
UTF-8
UTF16_BigEndian
UTF16_LittleEndian
UTF32_BigEndian
UTF32_LittleEndian
windows-10000
windows-10006
windows-10007
windows-10029
windows-10081
windows-1200
windows-1201
windows-1250
windows-1251
windows-1252
windows-1252
windows-1253
windows-1254
windows-1255

¿Por qué importa el texto?

- Porque **la mayor parte de la información** que las personas y las instituciones producen está en forma de texto.

- Porque la mayoría de los datos disponibles, y la mayoría de los datos que generan las organizaciones es **información no estructurada** (correos, documentos, reseñas, redes sociales, expedientes), no tablas ordenadas (como las del semestre pasado).

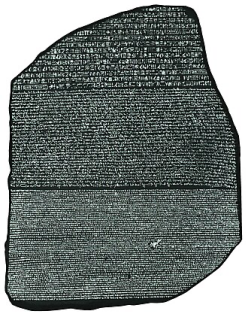
- Porque contiene **información que los números no capturan**.

- Porque ya **se puede medir con técnicas de procesamiento de lenguaje natural** y con el uso de herramientas de Inteligencia Artificial.



¿Por qué importa el texto?

- **El texto es el rastro de la vida social.** Leyes, discursos, sentencias, notas de prensa, entrevistas y publicaciones en las RRSS son la forma en que quedan registrados el poder, las normas y las opiniones de la sociedad en un momento determinado.
- **Permite estudiar lo que las encuestas no alcanzan.** Sirve para observar como habla la gente cuando nadie le pregunta, para reconstruir datos sin encuestas o para analizar actores que no responden cuestionarios, como legisladores, jueces o empresas.
- **El lenguaje mismo es objeto de estudio.** Cuando se encuadra un tema, qué palabras se usan para nombrar a un grupo o aquello que se omite y no se dice revelan temas de ideología, sesgos y relaciones de poder.



La librería stringr

- **Es parte del tidyverse y está construida sobre la vieja librería stringi**, que a su vez usa la biblioteca ICU (International Components for Unicode). Por lo que maneja bien Unicode y UTF-8.

- **Nombres consistentes:** todas las funciones empiezan con la partícula str_ y reciben el vector de texto como primer argumento, así que funcionan con el pipe (%>% o |>) y dentro de mutate() o filter().

- **Todo está vectorizado:** opera sobre el vector completo sin necesidad de ciclos, tanto en el texto como en el patrón.



Ejercicio práctico 01

- Abra el código de la clase y vaya resolviendo los ejercicios correspondientes.

```r
# Código para manipulación de texto.
# Sesión 11: Texto.
# Sesión práctica.

# Librerías ----
library(stringr)
library(stringi)
library(dplyr)
library(tidyr)

# Datos de trabajo ----

# Nombres de municipios escritos mal a propósito: espacios de más, mayúsculas
# inconsistentes, prefijos sueltos y acentos que a veces están y a veces no.
municipios_sucios <- c(
  "  Mpio. de Aguascalientes ",
  "AGUASCALIENTES",
  "Cd.  Juárez",
  "cd. juarez",
  "San  Luis   Potosí",
  "SAN LUIS POTOSI",
  "Mpio. Guadalajara",
  "Guadalajara  ",
```

El string va primero y todo empieza con str_

Tarea	Base R	stringr
Longitud	`nchar(x)`	`str_length(x)`
Cortar	`substr(x, 1, 2)`	`str_sub(x, 1, 2)`
Detectar	`grepl(patron, x)`	`str_detect(x, patron)`
Reemplazar	`sub(patron, nuevo, x)`	`str_replace(x, patron, nuevo)`
Mayúsculas	`toupper(x)`	`str_to_upper(x)`

Dos ventajas

Escribes str_ y el autocompletado te muestra todas. Y como el string va primero, funcionan con el operador %>%.

Debajo

stringr es una capa amable sobre stringi, que hace el trabajo pesado con ICU (la biblioteca de Unicode).

Para empezar

library(stringr) carga todo. Los datos de práctica fruit, words y sentences vienen incluidos.

“Escapar” un caracter.

"**Escapar**" es poner una diagonal invertida (\) antes de un carácter para que R lo trate como texto literal y que no active su función especial.

```
cita <- '"Vamos a conocer el hielo", le dijo Jose Arcadio a su hijo Aureliano'
cita <- "\"Vamos a conocer el hielo\", le dijo Jose Arcadio a su hijo Aureliano"
```

Si quisiera guardar un texto que tenga este símbolo: “\”, tengo que escapar ese símbolo para que R lo guarde tal cual viene escrito.

```
ruta <- "C:\Users\ana\datos.csv" # ERROR!
# Error: '\U' used without hex digits in character string (<input>:1:13)
ruta <- "C:\\Users\\ana\\datos.csv" # Bien :9
```

```
# print() muestra los escapes; writeLines() muestra lo que de verdad hay
print(cita)
writeLines(cita)
```

“Escapar” un caracter.

En una cadena de R solo dos caracteres necesitan escaparse obligatoriamente:

Carácter	Se escribe	Cuándo
Diagonal invertida	`\\`	Siempre
Comilla doble	`\"`	Solo si la cadena está delimitada con `"`
Comilla simple	`\'`	Solo si la cadena está delimitada con `'`

Aparte están las **secuencias de escape** que sirven para lo contrario: escribir caracteres que no puedes teclear directamente dentro de la cadena.

Secuencia	Significa
`\n`	Salto de línea
`\t`	Tabulador
`\r`	Retorno de carro
`\u00f1`	Carácter por su código Unicode (aquí, `ñ`)

Cualquier otra combinación da error: `"\d"` falla con `'\d' is an unrecognized escape`.

str_c() pega y propaga NA; **str_glue()** rellena campos

Ejercicio 2

```
clave <- "01"
nombre <- "Aguascalientes"

str_c("Clave ", clave, ": ", nombre)
#> [1] "Clave 01: Aguascalientes"

str_glue("Clave {clave}: {nombre}")
#> Clave 01: Aguascalientes

# Vectorizado: un resultado por elemento
str_c("Entidad ", c("01", "09", "15"))
#> [1] "Entidad 01" "Entidad 09" "Entidad 15"
```

La diferencia con paste()

```
# paste() convierte NA en texto
paste("Hola", c("mundo", NA))
#> [1] "Hola mundo" "Hola NA"

# str_c() lo propaga
str_c("Hola ", c("mundo", NA))
#> [1] "Hola mundo" NA
```

¿Cuál usar?

str_c() para pegar vectores. str_glue() cuando el texto es largo y las variables van en medio: se lee como una plantilla.

Idea clave

Un NA que se vuelve el texto "NA" es un error silencioso. str_c() lo deja como NA para que lo veas.

str_length() cuenta caracteres, **str_width()** columnas

Ejercicio 3

```
pais <- "México \U1F32E"   # México + taco
writeLines(pais)
#> México 🌮

str_length(pais)
#> [1] 8

str_width(pais)
#> [1] 9

# El taco es un carácter que ocupa
# dos columnas en pantalla.
```

str_length()

Número de caracteres (puntos de código Unicode). Es lo que quieres para validar: una CURP tiene 18, un código postal 5.

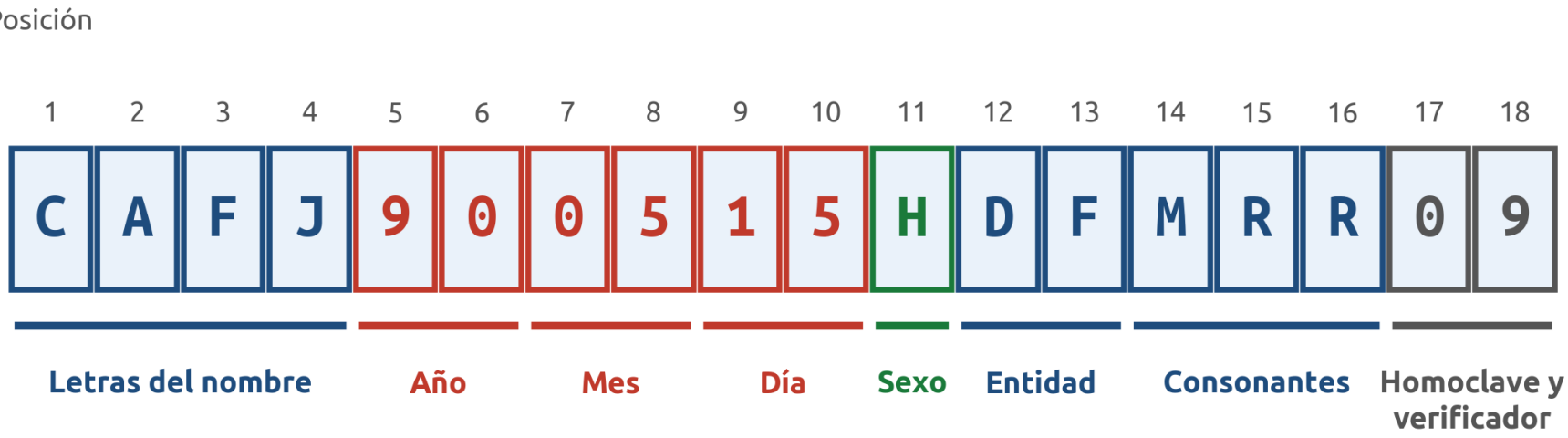
str_width()

Columnas que ocupa al imprimirse. Es lo que quieres para alinear tablas en consola o etiquetas de ancho fijo. Emojis y caracteres chinos miden 2.

Idea clave

Ambas funcionan sobre vectores completos: str_length(municipios) devuelve un número por municipio.

str_sub() corta por posición fija, como en la CURP




```
str_sub(curp, 5, 10)  ->  "900515"      str_sub(curp, 11, 11)  ->  "H"
```

Ejercicio 4

```
anio <- str_sub(curps, 5, 6)
mes  <- str_sub(curps, 7, 8)
dia  <- str_sub(curps, 9, 10)
sexo <- str_sub(curps, 11, 11)
```

Cuidado

str_sub() no valida nada. Con una CURP mal capturada devuelve basura sin avisar.

CURP ficticia con fines didácticos. Estructura oficial: RENAPO.

str_count() cuenta coincidencias de un patrón, no palabras

Ejercicio 5

```
# fruit viene con stringr: 80 frutas
vocales <- str_count(fruit, "[aeiou]")
head(vocales)
#> [1] 2 3 4 3 3 2

fruit[which.max(vocales)]
#> [1] "purple mangosteen"

# Para palabras: "una o más letras"
str_count("El texto es el dato", "\\w+")
#> [1] 5
```

Esto ya es un patrón

"[aeiou]" significa "una de estas cinco letras". No es texto literal.

Qué devuelve

Un entero por elemento. Sirve para filtrar (más de 4 vocales) o para estadísticas de texto (palabras por línea).

Idea clave

Contar palabras es contar un patrón.

Por eso str_count() es la puerta de entrada a las **expresiones regulares** 😈.

Expresiones regulares *(regex)*

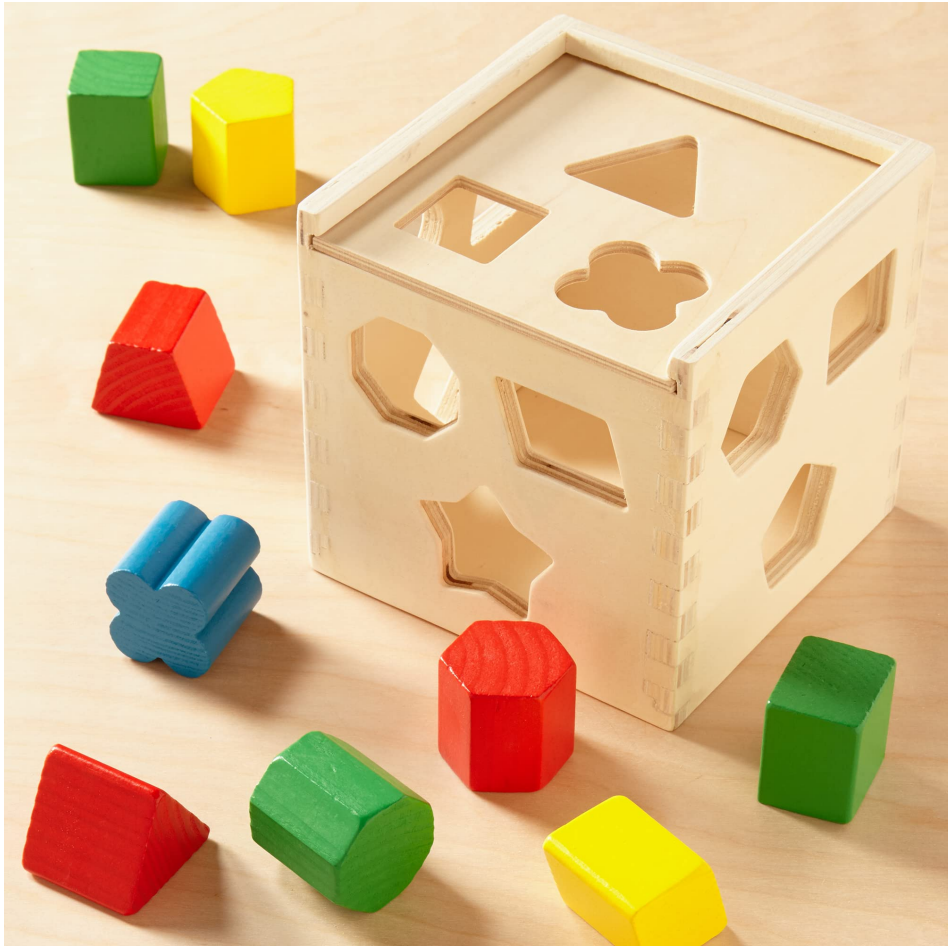
⭐Regex

Una expresión regular (*regex*) es **una cadena que describe un conjunto de cadenas**. Es un patrón escrito en una notación formal, y una cadena "coincide" si pertenece al conjunto que el patrón describe.

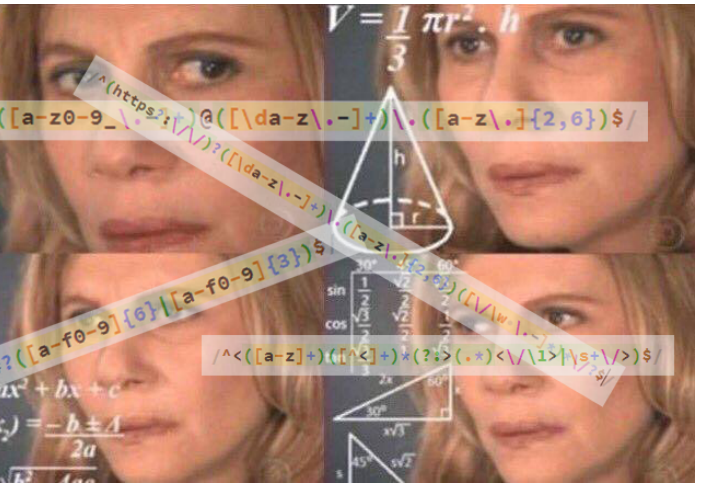
curp <- "^[A-Z][AEIOUX][A-Z]{2}\\d{2}(0[1-9]|1[0-2])(0[1-9]|[12]\\d|3[01])[HMX](AS|BC|BS|CC|CL|CM|CS|CH|DF|DG|GT|GR|HG|JC|MC|MN|MS|NT|NL|OC|PL|QT|QR|SP|SL|SR|TC|TS|TL|VZ|YN|ZS|NE)[B-DF-HJ-NP-TV-Z]{3}[A-Z0-9]\\d$"

tel <- "(?<![\\d+])(?:\\+?52[\\s.-]?(?:1[\\s.-]?)?)?(?:\\(?\\d{2}\\)?[\\s.-]?\\d{4}|\\(?\\d{3}\\)?[\\s.-]?\\d{3})[\\s.-]?\\d{4}(?!\\d)"

direccion_ip_valida <- "\\b(?:(?:25[0-5]|2[0-4]\\d|1?\\d?\\d)\\.){3}(?:25[0-5]|2[0-4]\\d|1?\\d?\\d)\\b"



Expresiones regulares *(regex)*















Una expresión regular (*regex*) describe un patrón, no un texto

`^`	inicio del string	`\d`	un dígito
`$`	fin del string	`\w`	letra, dígito o guion bajo
`.`	cualquier carácter	`\s`	un espacio en blanco
`[abc]`	uno de estos	`+`	uno o más
`[^abc]`	cualquiera menos estos	`*`	cero o más
`[a-z]`	un rango	`?`	cero o uno
`a|b`	a o b	`{3}`	exactamente tres
`(ab)`	grupo (se puede capturar)	`{1,2}`	entre uno y dos
		`\b`	frontera de palabra

`"^\\d{3}"` tres dígitos al inicio

`"berry$"` termina en berry

Con estas dieciséis piezas se resuelve el 90% de los casos

Lo demás se consulta cuando hace falta.

Otros patrones típicos:

Para encontrar	Patrón
Un número entero	`\\d+`
Código postal	`\\b\\d{5}\\b`
Año (1900 a 2099)	`\\b(19|20)\\d{2}\\b`
Fecha dd/mm/aaaa	`\\d{1,2}/\\d{1,2}/\\d{4}`
Monto en pesos	`\\$\\s?[\\d,]+(\\.\\d{2})?`
Teléfono de 10 dígitos	`\\b\\d{10}\\b`
Correo (versión simple)	`[\\w.+-]+@[\\w-]+\\.[\\w.-]+`
URL	`https?://\\S+`
Hashtag / mención	`#\\w+` / `@\\w+`
CURP	`[A-Z]{4}\\d{6}[HM][A-Z]{5}[A-Z0-9]\\d`
RFC de persona física	`[A-ZÑ&]{4}\\d{6}[A-Z0-9]{3}`
Palabra exacta	`\\bmigrante\\b`
Palabra con variantes	`\\bmigra(nte|ntes|ción)\\b`

La IA escribe el patrón y tú lo rompes a propósito

Ejercicio 16

```
# Pedido a un LLM: "regex para validar una CURP"
patron_curp <- "^[A-Z]{4}\\d{6}[HM][A-Z]{2}[B-DF-HJ-NP-TV-Z]{3}[A-Z0-9]\\d$"

es_valida <- str_detect(curps, patron_curp)
curps[!es_valida]
#> [1] "cafj900515hdfmrr09"   # minúsculas
#> [2] "LOPM851230XDFRRN02"   # sexo X
#> [3] "MARA0203HOCRRL07"     # fecha incompleta
```

1. Pedir

Describe el formato con palabras y un ejemplo. Pide el patrón en sintaxis estándar.

2. Probar en regex101

Pega el patrón y varios ejemplos. Mira qué coincide y qué no, pieza por pieza.

3. Pasar a R

Duplica cada diagonal invertida: \d se escribe "\\d". Prueba con str_view().

4. Romperlo

Dale casos que sabes que son inválidos. Si pasan, el patrón está mal aunque se vea bien.

Idea clave

Un patrón que acepta todo lo válido no sirve si también acepta lo inválido. El paso 4 es el que cuenta.

str_squish() quita todos los espacios sobrantes

Ejercicio 6

```
x <- "  San  Luis   Potosí  "

str_trim(x)
#> [1] "San  Luis   Potosí"

str_squish(x)
#> [1] "San Luis Potosí"

# Sobre el vector completo
municipios <- str_squish(municipios_sucios)
```

De dónde salen estas rarezas

Capturas a mano, copiar y pegar desde PDF, exportaciones de sistemas viejos con campos de ancho fijo o como resultado de otras funciones de *stringr*.

Primer paso recomendado

Antes de comparar, unir o contar, **pasa todo por str_squish()**. Es barato y evita horas de depuración.

Por ejemplo:

"Juárez" y "Juárez " son *strings* distintos para R.
Un espacio invisible puede romper un cruce de tablas.

str_remove() recibe patrones: el punto no es un punto

Ejercicio 7

```
str_remove("Cd. Juárez", "Cd. ")
#> [1] "Juárez"

# Funcionó... pero por suerte:
str_remove("CdX Juárez", "Cd. ")
#> [1] "Juárez"

# El punto significa "cualquier carácter".
# Para un punto literal, fixed():
str_remove(municipios, fixed("Mpio. de "))
```

Caracteres con poder

. ( ) [ ] { } + * ? ^ $ | \
Si el texto que quieres quitar tiene alguno, no es texto literal para R.

fixed()

Envuelve el patrón y le dice a stringr: compara tal cual, carácter por carácter. Además es más rápido.

Idea clave

str_remove(x, p) es str_replace(x, p, ""). Ambas reciben un patrón, no un texto.

str_replace() cambia la primera; **_all()** cambia todas

Ejercicio 8

```
str_replace("a-b-c", "-", " ")
#> [1] "a b-c"

str_replace_all("a-b-c", "-", " ")
#> [1] "a b c"

# Sin distinguir mayúsculas
patron_cd <- fixed("Cd. ", ignore_case = TRUE)
str_replace(municipios, patron_cd, "Ciudad ")
#> "Ciudad Juárez" "Ciudad Juárez" ...
```

Varios reemplazos a la vez

str_replace_all() **acepta un vector con nombres:**
c("Cd." = "Ciudad", "Mpio." = "Municipio")

Error típico

Usar str_replace() cuando hay varias coincidencias y revisar solo el primer renglón del resultado.

Idea clave

Si dudas, usa la versión _all().
Casi siempre es la que estás buscando.

str_unique() encuentra 8 municipios donde unique() ve 14

Ejercicio 9

```
length(unique(municipios))
#> [1] 14

# Ignora mayúsculas y acentos
str_unique(municipios, strength = 1)
#> [1] "Aguascalientes"  "Cd. Juárez"
#> [3] "San Luis Potosí" "Guadalajara"
#> [5] "Oaxaca de Juárez" "Mérida"
#> [7] "Tlalnepantla de Baz" "Ciudad Victoria"
```

Los argumentos

ignore_case = TRUE ignora mayúsculas. strength = 1 ignora además acentos (es un nivel del comparador de ICU).

Lo que no resuelve

"Cd. Juárez" y "Ciudad Victoria" siguen siendo dos escrituras distintas. Eso se arregla antes con str_replace().

Idea clave

Para saber cuántas categorías reales tiene una columna de texto, str_unique() con strength = 1 da la respuesta honesta.

str_split() parte; con **simplify = TRUE** da una matriz

Ejercicio 10

```
str_split("2026-10-05", "-")
#> [[1]]
#> [1] "2026" "10"   "05"

fechas <- c("2026-10-05", "2025-01-20")
str_split(fechas, "-", simplify = TRUE)
#>      [,1]   [,2] [,3]
#> [1,] "2026" "10" "05"
#> [2,] "2025" "01" "20"
```

Lista o matriz

Sin simplify devuelve una lista, porque cada elemento puede tener distinto número de piezas. Con simplify, una matriz.

str_pad() rellena con ceros

Ejercicio 11

```
claves_entidad <- c("1", "01", "9", "15")
str_pad(claves_entidad, width = 2, pad = "0")
#> [1] "01" "01" "09" "15"

claves_municipio <- c("1", "001", "39", "120")
str_pad(claves_municipio, width = 3, pad = "0")
#> [1] "001" "001" "039" "120"

# Clave geográfica INEGI: entidad + municipio
str_c(cve_ent, cve_mun)
#> [1] "01001" "01001" "09039" "15120"
```

El problema de siempre

Excel convierte "01" en 1. La capa de municipios de INEGI trae “01". Ante un *join* en esta situación, puede dar pie a que los estados cuya clave inicia con “0” no salgan en el mapa.

Otros usos

Alinear números en una tabla de texto (side = "left"), completar folios, armar identificadores de ancho fijo.

Idea clave

Una clave es texto, no número. Léela como character y garantiza su ancho con str_pad().

str_to_*() y str_wrap() trabajan las etiquetas

Ejercicio 12

```
str_to_title("SAN LUIS POTOSI")
#> [1] "San Luis Potosi"

str_to_sentence("OAXACA DE JUÁREZ")
#> [1] "Oaxaca de juárez"

str_to_upper("Mérida"); str_to_lower("MÉRIDA")
#> [1] "MÉRIDA"
#> [1] "mérida"
```

Ejercicio 13

```
largo <- "Tlalnepantla de Baz"

writeLines(str_wrap(largo, width = 12))
#> Tlalnepantla
#> de Baz

str_trunc(largo, width = 15)
#> [1] "Tlalnepantla..."
```

Ojo con las preposiciones

str_to_title() escribe "Oaxaca De Juárez". Para nombres oficiales conviene un diccionario.

Para mapas

str_wrap() parte etiquetas largas en dos líneas para que no se encimen; str_trunc() recorta en tablas.

str_detect() dice si coincide; **str_view()** muestra dónde

Ejercicio 14

```
tiene_berry <- str_detect(fruit, "berry")
fruit[tiene_berry]
#> [1] "bilberry" "blackberry" "blueberry" ...

str_view(fruit, "berry")
#> [3] │ bil<berry>
#> [4] │ black<berry>
#> ...

# Ahora: las que TERMINAN en berry.
# Con texto literal no se puede.
str_view(fruit, "berry$")
```

str_detect()

Devuelve TRUE o FALSE por elemento. Es el filtro: fruit[tiene_berry] o filter(str_detect(...)).

str_view()

Marca entre < > la parte que coincidió. Es la herramienta para probar patrones antes de usarlos.

El salto

"berry" busca ese texto. "berry$" busca ese texto al final. El $ no es texto: es un patrón.

str_match() da una columna por cada grupo capturado

Ejercicios 17 y 18

```
# E17: primera palabra de cada oración
str_extract(sentences, "^\\w+")
#> [1] "The" "Glue" "It's" "These" ...

# E18: capturar las piezas de la CURP
str_match(curps, "^([A-Z]{4})(\\d{2})(\\d{2})(\\d{2})([HM])")
#>      [,1]           [,2]   [,3] [,4] [,5] [,6]
#> [1,] "CAFJ900515H" "CAFJ" "90" "05" "15" "H"
#> [2,] "LOPM851230M" "LOPM" "85" "12" "30" "M"
#> [3,] NA            NA     NA   NA   NA   NA
#> [4,] "GARA020318H" "GARA" "02" "03" "18" "H"
```

Cada paréntesis es un grupo

Columna 1: la coincidencia completa. Columnas 2 en adelante: un grupo cada una, en orden.

Compara con str_sub()

La CURP en minúsculas dio "h" con str_sub(). Con str_match() da NA: te avisa que algo está mal.

Idea clave

str_sub() corta a ciegas. str_match() corta y valida al mismo tiempo. Para datos capturados a mano, la segunda.

Catorce tareas cubren el trabajo diario con texto

Pegar	`str_c(), str_glue()`	**Partir**	`str_split(), separate_wider_delim()`
Medir	`str_length(), str_width()`	**Rellenar**	`str_pad()`
Cortar por posición	`str_sub()`	**Mayúsculas**	`str_to_title()` y familia
Contar	`str_count()`	**Etiquetas**	`str_wrap(), str_trunc()`
Espacios	`str_squish(), str_trim()`	**¿Hay coincidencia?**	`str_detect(), str_view()`
Quitar y cambiar	`str_remove(), str_replace()` + `fixed()`	**Sacar la coincidencia**	`str_extract(), str_match()`
Únicos	`str_unique(strength = 1)`	**Sin mayúsculas**	`regex(ignore_case = TRUE)`

Lo que no vimos y sigue en los apuntes

Referencias a grupos previos (\\1), búsqueda hacia adelante (lookahead), clases Unicode (\\p{L}) y el paquete rebus del curso.

Gracias

¿Preguntas?